

A Novel Stateful Authentication Framework Approach With LLM-Based IDS for MQTT Security

Norziana Jamil¹, Member, IEEE, Mohd Shariq², Syed Shakir Hameed Shah³, Hala Shaker Mehdy, Zaid Abdi Alkareem Alyasseri⁴, Eghbal Hosseini, Aymen Dia Eddine Berini, and Zuhaira Muhammad Zain⁵

Abstract—Message queue telemetry transport (MQTT), a widely used messaging protocol in Internet of Things (IoT) applications, faces significant security challenges, particularly with denial-of-service (DoS) attacks and ensuring continuous, secure communication. Traditional security measures, like transport layer security (TLS)/secure socket layer (SSL), often introduce CPU overhead and bandwidth issues and require complex certificate management. This article has two main objectives: first, to provide a comprehensive review of MQTT security, detailing how MQTT operates, the associated security vulnerabilities, and existing solutions; and second, to propose a framework called the stateful authentication framework (SAF). SAF enhances MQTT security through two key components: the SAF authentication protocol and a large language model (LLM)-based intrusion detection system (IDS). The SAF protocol uses client-state information to establish secure, stateful interactions between clients and brokers and incorporates a multifactor authentication mechanism to prevent replay attacks and unauthorized access. Our evaluation of SAF shows that it significantly improves MQTT security, with Scyther simulations confirming the robustness of the protocol against known generic attacks. Additionally, the LLM-based IDS that integrates SAF's security policies provides an advanced approach for detecting intrusions. To our knowl-

edge, this is the first integrated approach combining a state-based authentication protocol and LLM-based IDS to tackle MQTT security challenges.

Index Terms—Authentication protocol, large language model (LLM)-based intrusion detection system (IDS), message queue telemetry transport (MQTT), MQTT security, stateful authentication framework.

NOMENCLATURE

MQTT Client/ broker	participating entities.
client_id	Client identifier.
client_state	Date and time of the session.
request_time	Date and time of the request.
identifier_msg	Unique message identifier.
t_msg	Timestamp.
k	Session key.
c	Counter.
ack	Acknowledgment.
h(·)	Secure one-way hash (SHA-256).
HMAC(·)	Hash-based message authentication code.
	Concatenation operator.
T _{oh}	Time to perform cryptographic hashing or HMAC.
T _{cd}	Time delay for local operation and communication.

I. INTRODUCTION

THE rapid expansion of the Internet of Things (IoT) ecosystem has introduced a myriad of new devices into the digital landscape, each requiring reliable and secure communication mechanisms to function effectively. Among the various communication protocols, message queue telemetry transport (MQTT) has emerged as a preferred choice due to its lightweight nature and efficiency in handling the resource constraints typically associated with Internet of Things devices [1]. MQTT operates on a publish/subscribe model on TCP [2], which allows devices to communicate asynchronously. This makes it highly scalable and adaptable for various IoT applications ranging from home automation to industrial IoT systems. However, with its growing adoption, MQTT has also become a target for cyber threats, particularly those aimed at disrupting service availability and compromising data integrity and confidentiality [3]. Traditional methods

Received 24 October 2025; revised 23 November 2025; accepted 14 December 2025. Date of publication 22 December 2025; date of current version 23 February 2026. This work was supported in part by Sharjah National Oil Company (SNOC), who served as the energy partner of the 5th Forum for Women in Research (QUWA): Together Innovating to Shape the Future at the University of Sharjah under Grant 12T052; and in part by Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia, through the Researchers Supporting Project under Grant PNURSP2025R763. (Corresponding author: Norziana Jamil.)

Norziana Jamil, Mohd Shariq, and Aymen Dia Eddine Berini are with the College of IT, United Arab Emirates University, Al Ain, United Arab Emirates (e-mail: norziana@uaeu.ac.ae; shariq99ansari@gmail.com; berini.aymen@uaeu.ac.ae).

Syed Shakir Hameed Shah is with the Institute of Informatics and Computing in Energy, College of Computing and Informatics, Universiti Tenaga Nasional, Selangor 43000, Malaysia (e-mail: syedshakir81@gmail.com).

Hala Shaker Mehdy is with the Institute of Informatics and Computing in Energy, College of Computing and Informatics, Universiti Tenaga Nasional, Selangor 43000, Malaysia, and also with the Department of Computer Science, Al-Mustansiriya University, Baghdad, Iraq (e-mail: halashaker2020@outlook.sa).

Zaid Abdi Alkareem Alyasseri is with the Department of Information Technology Research and Development Center, University of Kufa, Kufa, Iraq (e-mail: zaid.alyasseri@uokufa.edu.iq).

Eghbal Hosseini is with the Department of Applied Mathematics and Computer Science, Technical University of Denmark, 2800 Copenhagen, Denmark (e-mail: eghho@dtu.dk).

Zuhaira Muhammad Zain is with the Department of Information Systems, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, Riyadh 11671, Saudi Arabia (e-mail: zmzain@pnu.edu.sa).

Digital Object Identifier 10.1109/IJOT.2025.3646115

TABLE I
OVERVIEW OF MQTT

Reference	Research Contribution
Ref. [5]	The paper provides an overview of MQTT, including existing survey work, performance evaluation, applications, security issues, comparison to MQTT-SN, available tools and brokers, and a taxonomy of application layer protocols.
Ref. [6]	The paper presents a systematic review using PRISMA to examine research on the use of MQTT in IoT applications to enhance technology quality.
Ref. [7]	The paper investigates and classifies the available security methods used to address security issues and highlights the weaknesses of the current proposals. It also presents a set of guidelines for choosing the right security mechanism for different IoT applications.
Ref. [1]	The paper focuses on the root causes of MQTT security issues, the main threats, and corresponding offensive and defensive strategies, including machine learning, replay attacks, man-in-the-middle attacks, anomaly detection, blockchain-based trust mechanisms, Denial-of-Service (DoS) attacks, and encrypted transmission. It also discusses key MQTT security issues that should be addressed in the future.
Ref. [8]	The paper presents MQTT Quality-of-Service (QoS) performance.
Ref. [9]	The paper provides an overview of MQTT, including its architecture, message format, MQTT scope, and QoS for the MQTT levels.
Proposed	Our work herein provides a comprehensive review of the latest security challenges in MQTT, providing an understanding of how these security vulnerabilities in MQTT can be exploited, highlighting limitations of existing security countermeasures, and proposing an integrated approach as a novel framework with which to address the identified challenges.

for securing MQTT communications usually involve integrating external protocols such as transport layer security (TLS) or secure socket layer (SSL) [4]. While providing robust encryption and secure data transmission, these protocols also introduce significant overhead in terms of CPU usage and bandwidth consumption, which can create a huge burden on the performance of resource-constrained IoT devices. Furthermore, managing certificates for these protocols also introduces additional complexity.

There are also several studies focusing on strengthening MQTT security through lightweight authentication schemes or machine learning (ML)-based intrusion detection. However, these approaches typically operate as independent layers. Understanding these limitations, there is a need for more efficient solutions that can provide effective protection for MQTT communications without imposing excessive computational or operational burdens on IoT devices.

In this article, we present the stateful authentication framework (SAF), which integrates a state-based authentication protocol with a large language model (LLM)-driven intrusion detection system (IDS). The proposed framework enables context-aware anomaly detection and dynamic policy enforcement within MQTT environments. To the best of our knowledge, SAF represents the first unified architecture that combines authentication and intelligent intrusion detection for enhanced MQTT security.

A. Motivation

The growing ubiquity of IoT devices and the critical need for efficient communication protocols have led to extensive research on MQTT, with several review papers examining its various aspects over the years. This section presents a comparative analysis of the review papers and the distinctions outlined in our study, as shown in Table I. While the existing literature provides an understanding of the concepts of MQTT and its security challenges, this article aims to answer the following questions.

- 1) Is there any more recent threat to MQTT? What are they?
- 2) How can these threats be addressed with an integrated approach while maintaining the lightweight nature

of MQTT and its practical suitability for the IoT environment?

B. Our Contributions

Our contributions are summarized as follows.

- 1) We provide a comprehensive review of the recent security challenges inherent in the MQTT protocol. We systematically categorize and identify its vulnerabilities and existing approaches to mitigating threats, highlighting their limitations and requirements for further improvement.
- 2) We introduce the SAF, a novel approach that integrates two components to address various security challenges in MQTT, namely, the SAF authentication protocol and an LLM-based IDS.
- 3) We introduce a novel authentication protocol that uses a device's state as a way to identify legitimate MQTT devices. The SAF authentication protocol integrates device states and multifactor authentication to enhance the security of MQTT communications. The protocol not only addresses common vulnerabilities such as replay attacks and unauthorized access but also ensures that these enhancements do not compromise the protocol's lightweight and efficient nature. Our approach contrasts with traditional solutions that often rely on heavyweight protocols like TLS/SSL, which require high computational power and bandwidth.
- 4) We provide a detailed analysis of the SAF authentication protocol, including consideration and discussion of how the SAF authentication protocol addresses the majority of the security issues in MQTT systems. We also provide an efficiency analysis to demonstrate its practicality in a real MQTT system.
- 5) We provide a Scyther simulation of our proposed SAF protocol (Phases 1 and 2), which confirms that there is no active and passive security attack possible within bounds.
- 6) We introduce the concept of a large language-based IDS that complements the security solution to address security challenges in MQTT systems. We detail its design principles and the steps for integrating it into the overall SAF-MQTT system. We also highlight how

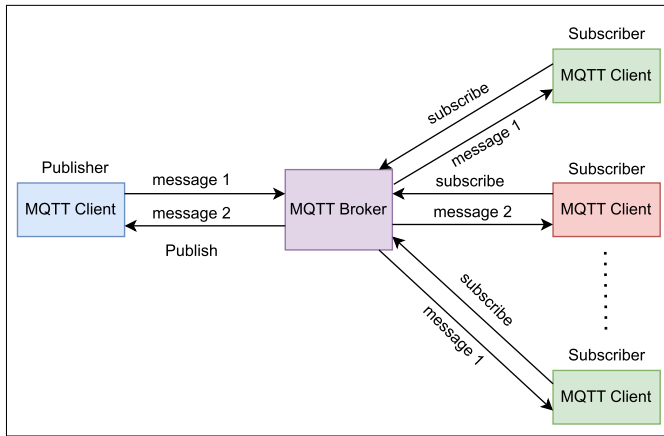


Fig. 1. Architecture of MQTT.

this approach is distinctive compared to existing IDS approaches.

C. Article's Organization

This article is organized as follows. Section II provides the preliminaries related to MQTT, a detailed review of the MQTT protocol, its core principles, operational flow, and common use cases within IoT environments. Section III discusses the security challenges associated with MQTT, including an in-depth analysis of vulnerabilities such as denial-of-service (DoS) attacks and potential data breaches. Section IV reviews recent security solutions to existing security challenges in the MQTT protocol. Section V introduces SAF, its components, and the novel approach it takes to enhance the security of MQTT communications. In this framework, all of its components are discussed extensively. The components include the SAF authentication protocol and the LLM-based IDS. Section VI presents the security analysis of SAF and Section VII presents the efficiency analysis for SAF authentication protocol. Section VIII presents the results of Scyther simulations, and finally, Section IX concludes this article by summarizing the key findings and discussing potential directions for future research to further improve MQTT security.

II. PRELIMINARIES OF MQTT

MQTT is one of the most widely adopted standards in the Industrial Internet of Things (IIoT). MQTT, a lightweight publish/subscribe (pub/sub) messaging protocol, is particularly effective for machine-to-machine (M2M) communication, which benefits from the asynchronous and decoupled nature of the pub/sub model [10]. This protocol was originally developed to facilitate the monitoring of oil pipelines via satellite, a scenario that demanded minimal bandwidth and low power consumption to ensure efficient data transmission over long distances with constrained resources. The OASIS technical committee standardized MQTT with the release of version 3.1.1 in 2014, in which the abbreviation for MQTT was omitted by the standardization group [11]. The latest version, MQTT 5.0 [12], released in 2019, brought significant improvements in scalability, making it well-suited for large-scale IoT systems and complex enterprise applications. As depicted in Fig. 1, the core architecture of MQTT consists

TABLE II
MQTT MESSAGE ARCHITECTURE

Field	Description
Message Type	Indicates the type of MQTT message (e.g., PUBLISH, SUBSCRIBE).
Duplicate Delivery	Flag shows whether the message is a duplicate.
QoS Level	Defines the level of service for message delivery.
Retain Flag	Instructs the broker to retain the message for new subscribers.
Topic Length	Specifies the length of the topic name.
Message Identifier	Unique ID for identifying the message.

of three primary components: publishers, subscribers, and brokers. Publishers are nodes that send messages, subscribers are nodes that receive messages, and brokers act as intermediaries that facilitate message distribution between publishers and subscribers. Brokers play a crucial role in managing and routing messages to ensure they reach the appropriate subscribers based on the topic and quality-of-service (QoS) levels.

A. Architecture of the MQTT

Each MQTT message begins with a fixed header, which is crucial for enabling the broker to appropriately manage and route the message to the subscribers.

The header contains details about the message type, topic, QoS level, and message flags. This fixed section ensures the standardized handling of messages across different MQTT implementations.

Following the header, the message includes a variable-length payload that contains the actual data being transmitted. The MQTT header fields encompass several key components: the message type, which specifies the nature of the message; the duplicate delivery flag, which indicates whether the message is being retransmitted; the QoS level, which defines the delivery guarantee; the retain flag, which instructs the broker to store the message for new subscribers; and the topic length, which denotes the length of the topic name associated with the message. A description of the MQTT message architecture is given in Table II.

B. Components of the MQTT Architecture

The MQTT architecture is designed to facilitate efficient communication between devices in IoT environments. The primary components are the client, MQTT broker, topic, publish, and subscribe. Table III summarizes the core components of the MQTT architecture and their respective roles.

C. Communication Flow in MQTT

The communication flow in MQTT is built around a publish/subscribe model, which provides a flexible and efficient mechanism for data exchange between devices. This model involves various steps, including connection establishment, message publishing, and subscription management. Table IV details the essential steps, message types, and their interactions in the MQTT communication process.

TABLE III
COMPONENTS OF MQTT ARCHITECTURE

Components	Description
Client	Initiates connections to the MQTT broker, publishes messages, and subscribes to topics. Can be a publisher, a subscriber, or both.
MQTT Broker	Central hub that manages message routing between clients. Handles connections, publishing, subscriptions, and message storage.
Topic	Structured path used for categorizing and routing messages. Allows efficient distribution of messages based on client subscriptions.
Publish	Process by which clients send messages to specific topics. Messages are forwarded by the broker to subscribers of the topic.
Subscribe	Process by which clients register interest in topics. Subscribers receive messages published to the topics they are subscribed to, with specified QoS levels.

TABLE IV
MQTT COMMUNICATION PROCESS

Step	Message Type	Sent by	Sent to	Description
1	CONNECT	Client	Broker	The client initiates a connection to the broker by sending a CONNECT message. This message includes client credentials and connection parameters.
2	CONNACK	Broker	Client	The broker responds with a CONNACK message, indicating whether the connection attempt was successful or failed.
3	PUBLISH	Client	Broker	Once connected, the client can publish messages to specific topics. This message includes the topic name and the payload (data).
4	(Optional) PUBACK	Broker	Client	If QoS level 1 is used, the broker sends a PUBACK message to acknowledge receipt of the published message.
5	SUBSCRIBE	Client	Broker	Clients send a SUBSCRIBE message to indicate their interest in receiving messages on specific topics. The message lists the topics and desired QoS levels.
6	SUBACK	Broker	Client	The broker responds with a SUBACK message to confirm the subscription request and indicate whether it was successful.
7	PUBLISH	Broker	Client	When a message is published to a topic, the broker forwards it to all clients that have subscribed to that topic.
8	UNSUBSCRIBE	Client	Broker	To stop receiving messages on a topic, a client sends an UNSUBSCRIBE message to the broker, specifying the topic to be unsubscribed from.
9	UNSUBACK	Broker	Client	The broker confirms the unsubscription with an UNSUBACK message.
10	DISCONNECT	Client	Broker	The client terminates the session by sending a DISCONNECT message to the broker, which then releases the resources associated with the session.

III. SECURITY CHALLENGES IN MQTT

This section reviews the critical vulnerabilities inherent in MQTT, focusing on key areas such as authentication, data integrity, confidentiality, and resilience against attacks.

A. Message Reordering

Message reordering in MQTT refers to the phenomenon whereby messages transmitted between clients and brokers do not arrive at the intended destination in the order they were originally sent. This issue can occur due to various factors, such as network congestion, varying message prioritization, or disruptions in the communication channel. In MQTT's publish/subscribe model, where messages are asynchronously published and subscribed to topics, maintaining the order of messages is crucial for ensuring the correct interpretation of data by subscribers. Message reordering can lead to significant disruptions in IoT applications relying on MQTT, potentially causing errors in data processing, incorrect decision-making based on outdated information, or functional failures in automated systems. Lo and Hsu [13] proposed a secure update method or framework to facilitate secure over-the-air (OTA) updates for IoT devices using the MQTT protocol, which could potentially be used to address the message ordering problem.

Im and Lim [14] proposed E-MQTT to improve MQTT by allowing publishers to set the number of responses they want to wait for, which reduces message exchanges and delays compared to traditional MQTT. This solution could potentially be used to address the message ordering problem. Wu and Li [15] proposed a new retain message storage mechanism in MQTT that prevents missing messages and repeated receipt of a message.

The message reordering problem is still quite challenging to address. Brokers may distribute a load across multiple instances, which can lead to situations where data sent in quick succession by a client are handled by different broker instances.

B. Interception

Interception refers to the unauthorized access or monitoring of messages exchanged between MQTT clients and brokers. A range of solutions have been proposed to address this interception problem. Chen et al. [16] suggest a blockchain-based access control scheme, which uses a decentralized blockchain to store access control information and a Patricia Trie-based adaptive priority topic search tree for efficient permission matching. Alotaibi et al. [17] introduced a secure enhance-

ment for MQTT using a distributed ML framework, enabling real-time monitoring and distributed detection of anomalous behavior. Paul et al. [18] focus on improving fault tolerance and tackling broker failure in MQTT through blockchain, addressing limitations in the current MQTT specification. Finally, Chen et al. [19] proposed a trust-based approach for data sharing in the MQTT environment, which computes trust ratings for brokers and uses them to control data flow.

These studies collectively offer a comprehensive set of strategies for enhancing the security and reliability of MQTT communication.

C. DoS Attack

In the context of MQTT, DoS attacks aim to disrupt the normal functioning of the MQTT broker or impair communication between MQTT clients by overwhelming the network or the broker with a high volume of malicious requests or traffic. These attacks can exploit vulnerabilities in the MQTT protocol's handling of connection requests, message queues, or resource allocation mechanisms. Haripriya and Kulothungan [20] introduced Secure-MQTT, a fuzzy logic-based IDS that dynamically generates rules to protect low-configuration devices. Dikii et al. [21] proposed a modular attack detection tool using classifiers based on username, device ID, and IP address, with the multilayer perceptron and support vector machine showing the best results. Syed et al. [22] developed an ML-based detection framework for the MQTT protocol, focusing on features such as field size and length to reduce false positives. Al-Fayoumi and Al-Haija [23] proposed a lightweight scheme based on ML that captures low-rate distributed DoS attacks based on the MQTT protocol in a software-defined IoT environment. Da Silva et al. [24] proposed the development of a platform based on the MQTT protocol for uncrewed aerial vehicle (UAV) control and DoS detection embedded in the UAV system. They demonstrated its efficiency for UAV control and that it guarantees the security of the communications with the broker and in the Wi-Fi UAV network.

Most of the proposed solutions are reactive, primarily responding after the attack happens. This approach is insufficient for proactively identifying and preventing DOS attacks before they can infiltrate and cause damage.

D. Resource Exhaustion

Resource exhaustion in MQTT refers to a type of security challenge where an MQTT broker or client becomes unable to function properly due to the depletion of its available resources. These resources can include CPU processing power, memory, network bandwidth, and even storage capacity. MQTT, being a lightweight protocol designed for constrained IoT devices, may be particularly susceptible to resource exhaustion attacks under certain conditions. Causes of resource exhaustion in MQTT include connection floods, publish floods, and malformed large messages. Da Paz et al. [25] highlighted the vulnerability of MQTT brokers to resource starvation attacks. They looked into how running out of resources affects an MQTT broker by carrying out a real attack that deliberately starved it of resources. They showed how information gathered from a simple physical

inspection can be used to make these resource-starving attacks more effective against three different types of MQTT brokers. Chen et al. [26] proposed a resource allocation strategy for mobile edge computing systems with hybrid energy harvesting, which could potentially be applied to MQTT to optimize energy usage. Viswanathan [27] examined the power consumption of the MQTT protocol by changing the QoS levels, adjusting payload sizes, and implementing authentication mechanisms. The study revealed that while the authentication mechanism does slow down data transfer from publishers to subscribers when using QoS 1, the average power consumption over time remains roughly the same, regardless of whether the authentication mechanisms are in place.

Most of the existing research on resource exhaustion in MQTT has highlighted significant vulnerabilities, including susceptibility to connection floods, publish floods, and malformed messages that can deplete critical resources such as CPU, memory, and bandwidth. However, not much attention has been devoted to creating a framework that uses less computational power but still keeps the data secure while resources face exhaustion.

E. Message Duplication

Message duplication in MQTT refers to the unintended repetition of messages, which can occur during the process of delivering messages from publishers to subscribers via an MQTT broker. The causes of message duplication vary and include QoS levels.

- 1) *QoS 0 (at Most Once)*: Messages are delivered once or not at all. There is no guarantee of message delivery, and no acknowledgment is required, which prevents duplication but also risks message loss.
- 2) *QoS 1 (at Least Once)*: Ensure that a message is delivered at least once. The publisher requires an acknowledgment from the broker for message delivery. If the acknowledgment is not received within a certain timeframe, the publisher may resend the message, leading to potential duplication.
- 3) *QoS 2 (Exactly Once)*: Provide the highest level of delivery assurance by ensuring that each message is received only once by the intended recipient. However, the complex handshaking process involved can still result in message duplication under certain failure conditions before the final acknowledgment is received and processed.

Broker failover can also cause message duplication. If an MQTT broker crashes and is restarted, it may reprocess certain messages or subscriptions, leading to duplication. In a distributed MQTT broker setup, load balancing and failover mechanisms can inadvertently cause messages to be sent more than once, as different brokers may independently attempt to ensure message delivery.

Wu and Li [15] improved the MQTT retain message mechanism to avoid message loss and repeated receiving problems. This was done to ensure that subscribers receive all missed messages. The improved system suggests storing more retained messages, which will consume more memory. Palmese et al. [28] introduced a dynamic QoS controller

operating on the broker. It receives specific performance requirements from subscribers for the MQTT-SN protocol. They found that QoS 2 in MQTT-SN offers the highest reliability for message delivery, with no packet loss and minimal data duplication. Matic et al. [29] came up with a new way to manage how messages are sent in an MQTT broker cluster. Their approach, called the resource-leveling forwarding algorithm, ensures that messages are directed to clients with enough capacity to handle them, which helps prevent message duplication. Pipatsakulroj et al. [30] created muMQ, an advanced MQTT broker that can manage many subscribers without losing any messages. It uses a special system to handle data more efficiently and a custom Internet protocol stack on top of a technology called the data plane development kit (DPDK). This setup could also contribute to reducing message duplication.

F. Privacy Risks

MQTT is designed to facilitate communication between devices in resource-constrained environments and operates on a publish/subscribe model wherein messages are sent to and received from specific topics. However, this model can expose sensitive information and compromise user privacy if not properly secured. MQTT does not inherently provide encryption for the data being transmitted. Without the implementation of additional security measures, such as TLS, messages are sent in plaintext. In MQTT, topics are hierarchical and can reflect the structure and function of the IoT system. Without proper access controls, external entities can infer sensitive information about the system architecture, device functions, or the nature of the data being communicated by observing the topic hierarchy and names. Additionally, an attacker who gains access to a topic can subscribe to it and receive all messages published to that topic, which can lead to the leakage of potentially sensitive information.

In cases where the content of the messages is encrypted, metadata such as the frequency and timing of messages, source and destination identifiers, and the size of the messages can be observed by adversaries. This metadata can be analyzed to infer sensitive information about the users or devices, such as their behavioral patterns or operational states. Several methods have been suggested to tackle privacy concerns in MQTT. Colombo et al. [31] introduced an access control framework that controls who can share data across different MQTT networks by using a decentralized way of enforcing rules. Markovic and Edwards [32] worked on making MQTT brokers more transparent by recording how the system behaves to ensure it follows data access rules. Fischer et al. [33] proposed ways to enhance privacy in MQTT. They tested a one-time password method that makes communication through MQTT topics completely hidden, helping to prevent user tracking and the creation of user profiles.

Collectively, these studies contribute to the development of strategies for addressing privacy risks in MQTT.

IV. REVIEW OF MQTT SECURITY COUNTERMEASURES

To address the security challenges outlined above, researchers have proposed several security countermeasures. This section reviews these evaluations, methods, and strategies.

A. TLS/Secure Socket Layer

TLS implementation enhances MQTT by providing encryption and authentication capabilities, securing the data exchanged between clients and brokers. It ensures the confidentiality, integrity, and authenticity of messages. The evolution and vulnerabilities of TLS and SSL are critical concerns in network security. In the analysis by Friesen et al. [34], the strengths and weaknesses of TLS were highlighted. A further study in [35] emphasized the challenges of authenticated encryption under TLS. Addressing the future threat of quantum computers, another study in Scott [36] proposed a post-quantum TLS implementation. The studies presented in [37] focused on TLS and SSL in cloud computing and wireless networks, respectively, stressing their security implications. Additionally, the research detailed in [38] explored quantum-resistant TLS for IoT devices, emphasizing the need for tunnel traffic detection. Vulnerabilities and countermeasures to SSL/TLS protocols were also discussed in [39], in which the direction needed for network security enhancement is given. In their assessment of TLS protocol security, Ivanov et al. [40] evaluated attack scenarios, compared TLS versions, and highlighted differences in TLS 1.3.3. While the study effectively explored various TLS protocol versions and their distinctions, it fell short of providing a comprehensive analysis of all potential attacks and vulnerabilities.

B. Authentication and Access Control

MQTT supports multiple authentication mechanisms, such as username/password, client certificates, and token-based authentication, enabling the verification of client identities and controlled access for authorized users. Access control lists (ACLs) further refine permissions, dictating which clients can publish or subscribe to specific topics. In [41], researchers introduced an alternative approach to MQTT security by discussing authentication and access control methods, imposing OAuth 2.0 for authorization and HMAC-based one-time passwords (HOTP) for authentication. This technique implements two-step authentication, achieves mutual authentication using OTP with a hash chain, and employs the advanced encryption standard (AES) for robust confidentiality protection. The key limitation is the increased complexity, which results in higher bandwidth usage due to the additional data exchange required for authentication. Each session involves sending and receiving authentication messages, adding overhead.

Harsha et al. [42] addressed MQTT vulnerabilities by proposing authentication and authorization techniques, including ACLs, to secure MQTT brokers. This approach emphasizes early attack prevention by integrating security mechanisms during initial broker setup, mitigates DDoS attacks and message flooding, and enhances overall security posture with intrusion detection system and intrusion prevention system (IDS/IPS) for real-time attack detection and prevention. The article faces two main limitations: it primarily focuses on authentication and ACLs as countermeasures, potentially neglecting other security aspects such as data encryption and integrity checks. Additionally, the use of the Shodan API may not fully capture the complexities and variations of real-world IoT environments.

Pahlevi et al. [43] introduced an event-based dynamic authentication protocol tailored for MQTT protocol brokers and constrained microcontroller devices. This approach reduces processing demands on resource-constrained devices. The protocol enhances MQTT security by improving the client registration phase and providing session-specific authentication properties. Two key limitations include the focus on authentication mechanisms, leaving other critical aspects insufficiently addressed, such as data integrity, confidentiality, and resilience to various attack vectors like man-in-the-middle attacks. Additionally, the testing environment does not fully replicate the diverse and dynamic conditions of real-world IoT deployments, potentially overlooking security vulnerabilities.

Buccafurri et al. [44] suggested a new way to authenticate MQTT using one-time passwords and the Ethereum blockchain. This method adds an extra layer of security to make sure both local and remote devices are verified. It also helps protect user privacy and ensures trust by using Ethereum smart contracts. One significant limitation regarding high bandwidth usage and complexity is that the proposed authentication mechanisms, including OTP and blockchain integration, introduce considerable data exchange and computational overhead. This increase in bandwidth usage and complexity can strain constrained IoT devices and networks, rendering the solution less feasible for environments with limited resources.

Pal and Jadidi [45] investigated how protocol-based and hybrid access control methods can effectively address the shortcomings of traditional access control mechanisms in IoT systems. A key limitation of their work is the heavy reliance on centralized trust anchors or servers for key management and policy enforcement, which introduces a single point of failure and scalability issues.

Kurdi and Thayanathan [46] discuss the challenges of creating a distributed architecture and implementing authentication schemes for the MQTT protocol using multiple brokers. They identified key issues such as scalable authentication, cryptographic methods, data integrity, and attack resistance. Their study identified two primary limitations in its methodology. First, the cryptographic method employed, based on 2048-bit RSA, is not suitable for resource-constrained devices due to its high computational and communication overheads. Second, the framework does not adequately address improving the authentication mechanism, which is critical to ensuring security in distributed MQTT architectures.

Han et al. [47] presented a way to manage who can access data in IoT systems using blockchain and advanced encryption techniques. To make sure that the data are secure and private, they used smart contracts on the blockchain to keep track of who has accessed the data and to prevent bad actors from getting in. To ensure that data remain safe and only accessible to the right people, all data encryption and decryption happen offline, meaning that the IoT devices do not need to stay connected to the Internet.

C. Message Payload Encryption

Numerous studies have investigated the encryption of MQTT data. It is crucial to comprehend encryption methods, particularly in the context of IoT devices with constrained

resources. Researchers have identified several drawbacks of these methods, including increased energy consumption, reduced processing speed due to large data sizes, and elevated network traffic. While maintaining data security, these factors impact the usability and efficiency of these technologies [48], [49]. Liao et al. [50] improved attribute-based encryption using synchronization to encrypt data to make MQTT more secure and protect messages. Numerous other studies, such as those presented in [51] and [52], have also concentrated on encrypting message payloads in MQTT. However, even with the growing focus on this issue, many proposed solutions still face challenges due to the high latency or extra load caused by cryptographic processing.

D. Rate Limiting

Rate limiting is a technique used in computer networks and systems to control the rate of traffic or requests that can be sent or received by a device, service, or user. It sets a predefined limit on how many times an action can occur within a specified timeframe. The objective of rate limiting is to prevent resource exhaustion, ensure fair usage of network resources, mitigate DoS attacks, and maintain overall system stability and performance [53].

Kumar and Vasthimal [54] and Nawab and Kulkarni [55] proposed specialized rate-limiting systems designed for managing time series data and distributed environments, respectively. La Marra et al. [56] added usage control to their system's workflow, enabling the close monitoring and control of what subscribers can do with the data over time, based on changing factors related to the subscriber, the environment, or the data themselves.

Further investigating how these solutions are integrated into various MQTT-based applications and evaluating their practicality and effectiveness in different contexts will be valuable.

E. Intrusion Detection System and Intrusion Prevention System

An IDS is a cybersecurity mechanism designed to monitor, analyze, and detect unauthorized or malicious activities within a network or system to prevent potential security breaches. It plays a crucial role in identifying threats such as unauthorized access, data manipulation, or DoS attacks by continuously evaluating system logs and network traffic patterns. Traditional IDSs use signature-based or anomaly-based methods, but these often struggle to detect novel or evolving cyber threats in dynamic and resource-constrained environments like the MQTT protocol, widely used in the IoT. MQTT's lightweight nature and limited security features make it particularly vulnerable to attacks such as spoofing, message tampering, and DoS.

In this context, LLM-based IDS approaches have emerged as a promising solution, leveraging advanced natural language processing and contextual learning capabilities to detect complex, previously unseen attack patterns more effectively. By understanding data semantics, behavioral patterns, and contextual relationships within MQTT communications, LLM-based IDSs significantly enhance detection accuracy, reduce

false positives, and provide adaptive, intelligent protection for modern IoT and MQTT-driven environments.

Ciklabakkal et al. [57] and Mosaiyebzadeh et al. [58] developed IDPSs that use ML techniques. Birkinshaw et al. [59] introduced a software-defined network to strengthen defenses against DoS and port scanning attacks. In [60], an MQTT parsing engine was designed that checks MQTT messages for issues and can be added to existing network defenses to catch problems early by thoroughly examining the details of each message. Meanwhile, Hindy et al. [61] tested six different ML techniques to spot attacks on MQTT systems, showing that using data flow patterns helps identify MQTT attacks better than traditional methods, which rely on individual data packets. Mahmood et al. [62] proposed an ML-based IoT security framework enhanced with an LLM. The system integrates an isolation forest for anomaly detection and a random forest for device integrity assessment. To improve interpretability, the outputs are further analyzed by GPT-4o mini, which generates statistical summaries, risk scores, and human-readable explanations. This reduces the semantic gap, supports efficient decision-making, and enables nontechnical users to understand and act on security alerts without relying on expert analysts. Elouardi et al. [63] proposed a survey based on an IDS that plays a critical role by identifying and responding to various IoT-based attacks. A wide range of IDS techniques spanning ML, deep learning (DL), and LLMs are currently employed to detect anomalies and intrusions within IoT data. Despite these advancements, there remains considerable room for improvement in detection accuracy and robustness, and its integration with IoT communication protocols such as MQTT.

V. SAF FOR MQTT

In this section, we introduce an integrated approach, an SAF for MQTT, referred to as SAF. Our framework comprises two components: the SAF authentication protocol and the SAF LLM-based IDS.

A. SAF Authentication Protocol Description

The notations and definitions used in our protocol are given in Nomenclature. The SAF authentication protocol consists of the following two phases—Phase 1: pre-session and Phase 2: in-session. The steps of the protocol are as follows.

1) *Phase 1: Pre-Session (Conducted in a Controlled Secure Environment)*: In this phase, an offline registration takes place. Prior to starting the offline registration, the MQTT client determines the client identifier, `client_id`, such as the MAC address, which will then be recorded by the broker. After this is done, the following steps are executed.

- 1) The MQTT client requests the MQTT broker to establish a client state by presenting its `client_id`. The MQTT client records the `request_time` (date and time of the request) as “Level 1 information.”
- 2) The MQTT broker verifies the `client_id`. Upon successful verification, proceed to the next steps. Otherwise, offline registration is repeated.
- 3) The MQTT broker creates the client state, `client_state`, which includes information such as the `session_time`

(time and date of the session) and the estimated duration of the pre-session.

- 4) The MQTT broker computes the hash value of the `client_state`, denoted as x .
- 5) MQTT broker generates a session key k and a counter c . The length of the session key is 32 bytes of cryptographically random value, and the counter is 2 bytes long, incrementing by 1 for each new session established.
- 6) The MQTT broker computes $y = x || k || c$ and sends y to the MQTT client. This indicates a secure and successful establishment of the client state by the MQTT broker.
- 7) The MQTT client stores x , k , and c , and generates an acknowledgment ack.
- 8) The MQTT client sends ack back to the MQTT broker to confirm receipt.

All steps in Phase 1 are shown in Algorithm 1, and the schematic of Phase 1 is depicted in Fig. 2.

Algorithm 1 Pre-Session Authentication Protocol for MQTT

```

1: Step 1 : Client requests client's state
2: client_id  $\leftarrow$  “ClientMAC123”
3: request_time  $\leftarrow$  “2023 – 06 – 14T10 : 00 : 00Z”
4: mqtt_client.send_request(client_id,request_time)
5: Step 2 : Broker verifies client
6: if mqtt_broker.verify_client(client_id) then
7: Step 3 : Broker Establishes Client-State
8: client_state  $\leftarrow$  mqtt_broker.create_client_state
9: (session_time = “2023 – 06 – 14T10 : 05 : 00Z”, estimated_duration = “30 mins”)
10: Step 4 : Compute hash of client's state
11: h_client_state  $\leftarrow$  hash(client_state)
12: Step 5 : Generate session key and counter
13:  $k \leftarrow \text{generate\_session\_key}() \triangleright 32 \text{ bytes}$ 
14:  $c \leftarrow 0 \triangleright \text{Initial counter}$ 
15: Step 6 : Send client's state information to client
16: mqtt_broker.send_to_client(h_client_state,k,c)
17: Step 7 : Client acknowledges receipt
18: mqtt_client.store_credentials(h_client_state,k,c)
19: ack  $\leftarrow$  “Acknowledged”
20: mqtt_client.send_ack(ack)
21: else
22: Verification failed, repeat offline registration
23: end if

```

2) *Phase 2 [In-Session (Over an Insecure Channel)]*: It is given as follows.

- 1) The MQTT client determines the Level 1 information, which includes details from the last communication session. The first session includes the pre-session setup date and time. For subsequent sessions, it includes the previous session's date and time.
- 2) The MQTT client initiates a session to either publish data or subscribe to a topic.
- 3) The MQTT broker requests the `client_state` and Level 1 information from the MQTT client.
- 4) The MQTT client computes α , which is the HMAC value of the concatenation of `client_state` and c , using the session key k and includes a timestamp t_{msg} . The

MQTT client sends α with t_msg and a unique message identifier $identifier_msg$ to the MQTT broker.

- 5) The MQTT broker computes the HMAC value using the received $client_state$, c , and k to verify the authenticity of the client. The broker checks the validity of t_msg and $identifier_msg$ to ensure the message is fresh and has not been duplicated or replayed. Note that the following conditions hold.
 - a) The broker checks t_msg to ensure that it falls within an acceptable time window.
 - b) This prevents replay attacks where an attacker might capture and resend a previously valid message at a later time.
 - c) The broker maintains a record of the $identifier_msg$ values it has processed for the current session.
 - d) Upon receiving a new message, the broker checks whether the $identifier_msg$ has previously been seen in the current session.
 - e) If $identifier_msg$ is new and has not been processed before, the broker proceeds with further validation and processing.
 - f) If the $identifier_msg$ has already been processed (indicating a duplicate message), the broker rejects the message as it could be a replay attack or an unintended duplication.
 - g) The MQTT broker sends a verification status to the MQTT client. If verification is successful, the MQTT client is allowed to send data. Otherwise, the MQTT client is denied data transmission.
- 6) The MQTT broker sends the verification status to the MQTT client. For data requiring high protection, the additional Step 7 can be performed.
- 7) The MQTT client encrypts the data using the session key k and the ASCON scheme [64] to authenticate-then-encrypt the data. This ensures that the data are authenticated and encrypted before transmission.

Algorithm 2 shows all the steps for Phase 2, and Fig. 3 illustrates Phase 2 of the SAF authentication protocol diagrammatically.

To ensure that the authentication framework for MQTT is both practical and secure, the following requirements must be fulfilled.

- 1) Both MQTT publishing clients and MQTT subscribing clients must establish a client state with the MQTT broker. This state records information about the historical interactions between the MQTT broker and its clients. The process involves the MQTT clients initiating communication with the MQTT broker for a short duration. The information recorded includes the date and start time of the initial communication, its duration, and the client identifier (e.g., IP address or MAC address). The states are then securely hashed (e.g., using a 256-bit hash value) and stored in the MQTT broker. This means that the MQTT broker maintains records of all client states and assumes client identification through the validity of these states. The client state serves as fixed information.
- 2) For secure authentication, multifactor authentication (MFA) is integrated into SAF. The MFA has two levels.

Algorithm 2 During Session Authentication Protocol for MQTT

```

1: Step 1 : Determine Level 1 Information
2: if is_first_session() then
3:   level_1_info ← pre_session_time ▷Timestamp from Phase 1
4: else
5:   level_1_info ← previous_session_time ▷Timestamp from the last session
6: end if
7: Step 2 : Initiate Session
8: mqtt_client.initiate_session()
9: Step 3 : Broker Requests Client-State and Level 1 Information
10: broker_request ← mqtt_broker.request_client_state_and_info()
11: Step 4 : Compute and Transmit HMAC
12: client_state ← mqtt_client.get_client_state()
13: t_msg ← get_current_timestamp() ▷Example current timestamp
14: identifier_msg ← generate_unique_identifier()
   ▷Generate unique identifier for the message
15:  $\alpha$  ← compute_hmac(client_state||c,k)
16: mqtt_client.send( $\alpha$ , t_msg, identifier_msg)
17: Step 5 : Verify HMAC and Client-State
18: (received_α, received_t_msg, received_identifier_msg) ← mqtt_broker.receive_hmac_and_info()
19: if mqtt_broker.verify_α_and_state(received_α, client_state, c, k) and mqtt_broker.verify_timestamp(received_t_msg) and mqtt_broker.verify_identifier(received_identifier_msg) then
20:   verification_status ← "Approved"
21: else
22:   verification_status ← "Denied"
23: end if
24: Step 6 : Send Verification Status
25: mqtt_broker.send_verification_status(verification_status)
26: Step 7 : Encrypt Data (Optional)
27: if requires_high_protection() then
28:   data ← mqtt_client.get_data()
29:   encrypted_data ← ascon(k, data)
30:   mqtt_client.send_encrypted_data(encrypted_data)
31: end if

```

- a) *Level 1*: Presentation of dynamic and validated information, such as the information about the last communication (publishing, subscribing, or requesting).
- b) *Level 2*: Involve a security mechanism that cannot be easily compromised.

- 3) All cryptographic primitives in the SAF authentication protocol are assumed to be secure as per current implementation, to provide adequate security (reflected in key lengths and hash values) for IoT environments. The

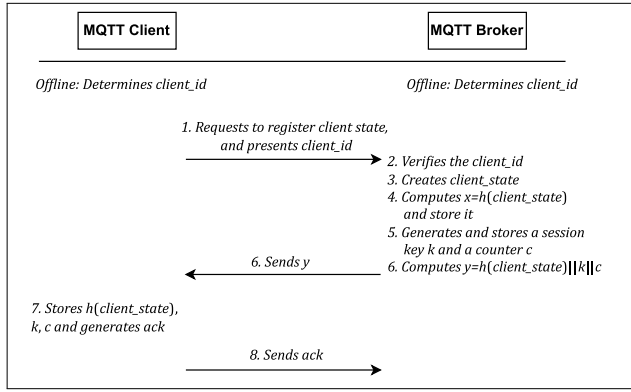


Fig. 2. Phase 1 of SAF.

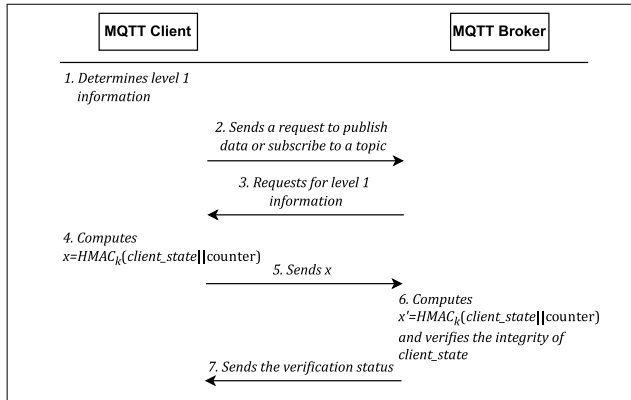


Fig. 3. Phase 2 of SAF.

ASCON scheme [64] is proposed to authenticate and then encrypt data that requires high protection.

- 4) The size of the counter is determined by the expected maximum number of data points sent by the MQTT client. In this article, we propose the size of the counter to be 2 bytes. A 16-bit counter can represent 216, which counts up to 65 536 data points.

B. LLM-Based IDS for SAF

We propose an LLM-based IDS that is integrated with the SAF authentication protocol. The security policies for SAF are defined to monitor all MQTT communication and detect any potential intrusion or malicious activity. These policies also constitute rate limiting as follows.

- 1) The MQTT broker limits the number of MQTT publishing and subscribing clients.
- 2) The MQTT broker limits the number of sessions that MQTT publishing and subscribing clients can have in a day (duration is set by the MQTT broker).
- 3) No new registration of MQTT client or broker is allowed for a defined time.
- 4) Each MAC address is restricted to a single entity if there is simultaneous communication from the same.

The high-level architecture of our IDS is given in five steps and is shown in Fig. 4. The steps are as follows.

- 1) *MQTT Data Ingestion*: The system collects MQTT messages, which are text-based and contain unstructured

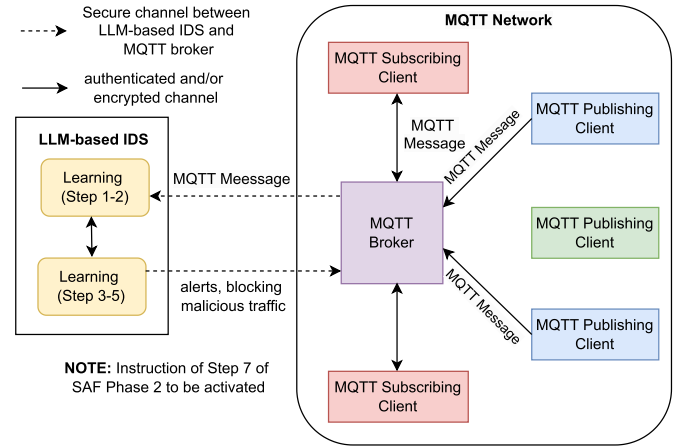


Fig. 4. Architecture of LLM-based IDS in MQTT.

data such as topics, sensor readings, commands, or status updates.

- 2) *Preprocessing*: After collecting MQTT messages, the system converts MQTT message payloads into a format that the LLM can analyze. This involves tokenizing the text and encoding it into a vectorized form.
- 3) *LLM Model*: The system then uses an advanced language model such as GPT-4 to process and understand the semantics of the MQTT messages. This is to allow the model to detect complex, context-based anomalies that might be indicative of security threats such as DoS attacks.
- 4) *Pattern Detection*: The system then identifies unusual patterns or anomalies within the text of MQTT messages, such as unexpected commands or unusual data payloads.
- 5) *Alert Generation*: If a pattern or anomaly is detected, the system generates an alert that can prompt further investigation or automated responses to potential security threats. For the sake of efficiency, if an alert is generated, Step 7 of Phase 2 of the SAF authentication protocol should be executed.

The concept of LLM-based IDS that we propose here leverages advanced natural language processing capabilities to analyze and understand the context and semantics of MQTT messages based on the security policy set earlier.

Fig. 4 shows that the MQTT broker serves as a central hub where all the other MQTT clients connect. The green colored arrow indicates that the communication is authenticated, where the SAF authentication protocol is applied, while the blue colored arrow indicates a secure channel between the LLM-based IDS and the MQTT broker through any affordable secure protocol. The MQTT messages refer to message topic (submitted by MQTT subscribing clients), payload size, QoS levels, and frequency of messages. The alerts or decisions returned to the MQTT broker include abnormal message rates, suspicious contents, and/or unauthorized topic access. Should the alert be sent to the MQTT broker, Step 7 of Phase 2 of the SAF authentication protocol will be executed.

The primary difference between our LLM-based IDS and traditional ML-based IDS for the MQTT protocol lies in the approach to handling and interpreting data. In our LLM-

TABLE V
COMPARISON OF MEAN AND THROUGHPUT FOR ENCRYPTION/DECRYPTION OPERATIONS IN ASCON AND AES VARIANTS [65]

Algorithm	Mean Enc Time (μ s/256B)	Enc Throughput (B/s)	Mean Dec Time (μ s/256B)	Dec Throughput (B/s)
AES-128-CTR	1,910	134,030	1,906	134,250
AES-128-GCM	3,555	72,012	3,558	71,947
AES-256-CTR	2,668	95,948	2,665	96,060
AES-256-GCM	3,930	65,130	3,933	65,078
ASCON-128	422	606,664	470	543,927
ASCON-128a	319	802,068	371	689,934
ASCON-80pq	440	581,693	493	518,826

based IDS, the LLM leverages advanced natural language processing capabilities to analyze and understand the context and semantics of MQTT messages. This will enable the detection of sophisticated threats and anomalies that traditional ML models might miss. Cyber attacks are becoming more advanced, and the LLM-based IDS will be able to parse complex data patterns within MQTT payloads that could contain these advanced attacks, i.e., context-dependent attacks. In contrast, ML-based IDS focuses on statistical patterns and anomalies and typically relies on predefined feature extraction and pattern recognition techniques, which means they struggle with contextually complex threats.

The proposed SAF focuses on state tracking for maintaining session continuity and integrity between client and broker, assuming ordered message flow. We plan to extend SAF by integrating a sequence-aware state synchronization mechanism, where each authentication and message exchanged is tagged with a specific session sequence number and timestamp. Thus, the disordered distribution by distributed brokers will be detected and correctly reordered.

An attribute-based access control mechanism that can authorize clients dynamically based on attributes such as device type, trust level, and state history can be introduced. In addition, an anonymization layer can be introduced to further obfuscate sensitive topics based on cryptographic hashing and tokenization before the sensitive information is published.

VI. SECURITY ANALYSIS OF SAF

In this section, we analyze the security of the proposed SAF for MQTT. The analysis focuses on how SAF addresses various security threats and meets the requirements for a secure IoT communication protocol.

A. Authentication and Identity Verification

First, the client-state mechanism ensures that each MQTT client has a unique state recorded by the MQTT broker to reflect the client's interaction history. We propose a minimum of a 256-bit hash value that represents the client state; this is strong enough to resist a collision attack. Including details such as the date, start time, and duration of the initial communication provides a basis for verifying client authenticity. The stateful nature of the communication, with each session requiring dynamic but specialized data, makes it difficult for an attacker to get themselves into the communication stream undetected. Second, the broker checks the validity of t_msg (timestamp) and $identifier_msg$ (unique message identifier) to ensure the security and integrity of the MQTT communication. If the broker encounters two different t_msg values that have the same $identifier_msg$, this indicates a potential security breach. This situation suggests that either the same message has been

sent twice (possibly due to network issues or client-side errors) or an attacker is attempting to replay a previously captured message with the same identifier. In this case, upon detecting this discrepancy, the broker should reject the message. It can log this event for auditing purposes and potentially implement additional security measures (such as temporarily blocking further messages from the client or alerting administrators). To prevent message duplication and replay attacks, using unique identifiers ($identifier_msg$) for each message and checking their uniqueness within a session helps mitigate these risks. Additionally, timestamps (t_msg) ensure that messages are processed within a valid time frame.

SAF employs multifactor authentication (MFA) in a different way to suit IoT environments. Level 1 involves the use of dynamic information. In SAF, the dynamic information is the last MQTT client communication session with the MQTT broker. This creates additional difficulty for the attackers as they need to know recent communication patterns to pass the authentication. Level 2 in MFA involves a mechanism that is difficult to compromise, and in SAF, the mechanism uses the ASCON scheme [64] to provide authenticated encryption alongside the MQTT client's session key and counter.

The use of ASCON instead of AES is supported by its suitability for resource-limited environments where low latency, energy efficiency, and long-term security are important. Table V presents a comparative analysis of ASCON and AES variants. ASCON provides strong protection against common attacks and performs well on devices with limited processing power and memory. Since ASCON has been standardized by NIST as part of its lightweight cryptography portfolio, it offers a reliable and tested option for secure communication in IoT and cyber-physical systems. This makes it a practical choice for achieving both efficiency and security within the proposed framework.

B. DoS Prevention Through MQTT Client and Broker Restrictions

SAF introduces several restrictions to enable withstanding various types of threats, such as DoS attacks and message reordering problems. First, it reduces the risk of resource exhaustion attacks such as DoS by limiting the number of MQTT clients (both publishers and subscribers) connected to the MQTT broker, and the maximum number of sessions that MQTT clients can have in a day or a duration determined by the MQTT broker. This means that any malicious activity that overloads the MQTT broker with excessive data connections can be detected. SAF also prohibits new client or broker registrations to prevent unauthorized changes in the network that could be exploited for attacks. To prevent MAC

spoofing attacks, SAF restricts each client identifier to only correspond to a single entity and aborts communication if multiple connections are detected.

SAF introduces the session key, combined with an incremental counter. At least two challenges are identified in this setting. First, the counter was independently and randomly generated by the MQTT broker in a controlled and secure environment. The guessing space is massive. Second, the counter is incremented in each session. In any random session, it is very difficult for an attacker to guess or deduce the number of sessions that have already taken place in order to determine the current counter value.

In terms of replay attacks, SAF requires a valid client state for validation. The requirement of a valid client state with Level 1 information makes it difficult for an attacker to launch replay attacks because they cannot reuse the data to gain unauthorized communication. Because each session uses a unique, incrementing counter, it ensures that old session data cannot be reused.

C. Mitigating Message Duplication in MQTT

SAF implements stateful tracking mechanisms to monitor and record the state of each message delivered between the client and the broker. This includes tracking which messages have been sent, acknowledged, and processed. SAF can distinguish between new messages and duplicates by maintaining detailed records of message IDs and their statuses. Each message sent through MQTT is tagged with a unique identifier and timestamp. SAF uses these identifiers to keep track of the state of each message. For instance, if a message with the same identifier is received more than once, SAF recognizes it as a duplicate and prevents it from being processed again.

In terms of QoS, SAF improves it as follows.

- 1) *QoS 1*: SAF tracks each message's delivery status and acknowledgment at a more granular level. It ensures that a message is not resent unless it is confirmed that the previous instance of the message was not acknowledged. This can reduce the chances of unnecessary message retransmissions and subsequent duplications.
- 2) *QoS 2*: To ensure that each message is processed only once, SAF uses unique message identifiers and verifies the final acknowledgment to prevent the same message from being mistakenly processed multiple times in case of network failures or retries.

VII. EFFICIENCY ANALYSIS

To analyze the efficiency of the SAF authentication protocol in terms of time, the operations involved in each phase are broken down. We assume that the time units are relative and focus on comparing the computational efforts across phases. The notation used in this section is given in Nomenclature, and the unit times for Phases 1 and 2 are given in Tables VI and VII, respectively. The whole efficiency is now analyzed as follows: a) total time (Phase 1): $2T_{oh} + 4T_{cd}$ and b) total time (Phase 2): $2T_{oh} + 1T_{cd}$. Therefore, the total time units for the SAF authentication protocol are $4T_{oh} + 5T_{cd}$. This reflects the cumulative time required for cryptographic operations T_{oh} and local processing delays T_{cd} across both phases of the protocol.

TABLE VI
TIME ANALYSIS OF PHASE 1

Steps	Time
Offline registration	$1T_{cd}$
Client registration	$1T_{cd}$
Client requests client-state	Negligible (assumed 0)
Broker verifies client	$1T_{oh} + 1T_{cd}$
Broker establishes client-state	$1T_{oh} + 1T_{cd}$
Send client-state information to client	Negligible
Client acknowledges receipt	Negligible

TABLE VII
TIME ANALYSIS OF PHASE 2

Steps	Time
Determine Level 1 information	Negligible
Initiate session	Negligible
Broker requests client-state and Level 1 information	Negligible
Compute and transmit HMAC	$1T_{oh}$
Verify HMAC and client state	$1T_{oh} + 1T_{cd}$
Send verification status	Negligible

Scyther results : verify					
Claim				Status	Comments
Proposed, MQTTClient	Proposed, MQTTClient1	Secret x		ok	No attacks within bound
	Proposed, MQTTClient2	Secret y		ok	No attacks within bound
	Proposed, MQTTClient3	Niagree		ok	No attacks within bound
	Proposed, MQTTClient4	Nisynch		ok	No attacks within bound
	Proposed, MQTTClient5	Alive		ok	No attacks within bound
	Proposed, MQTTClient6	Weakagree		ok	No attacks within bound
MQTTBroker	Proposed, MQTTBroker1	Secret Hash(clientstate)		ok	No attacks within bound
	Proposed, MQTTBroker2	Secret Concatate(Hash(clientstate),c,k)		ok	No attacks within bound
	Proposed, MQTTBroker3	Niagree		ok	No attacks within bound
	Proposed, MQTTBroker4	Nisynch		ok	No attacks within bound
	Proposed, MQTTBroker5	Alive		ok	No attacks within bound
	Proposed, MQTTBroker6	Weakagree		ok	No attacks within bound
Done.					

Fig. 5. Scyther simulation results of Phase 1.

VIII. SECURITY VERIFICATION USING SCYTHY SIMULATION TOOL

This section presents a formal security verification using the Scyther simulation tool, which is mainly used for analyzing cryptographic security protocols [66], [67], [68], especially ensuring security properties such as secrecy and authentication among participating entities. To perform the Scyther simulation of our proposed protocol, we first install the necessary prerequisites: Python 2.7, wxPython 2.8, Graphviz 2.46, and Scyther 1.1.3. Next, we set up the experimental environment on a 64-bit Linux system running Ubuntu 21.10, with an AMD Ryzen 9 5900HX processor at a frequency of 4.6 GHz and 12 GB of RAM. The Scyther tool supports both symmetric and asymmetric cryptographic operations, such as digital signatures and Diffie–Hellman key exchanges. The protocol can be specified using a Security Protocol Specification Language called SPDL, where roles, messages, and cryptographic operations can be defined. The Scyther also verifies the correctness of security protocols as well as checks whether the protocol is secure against several known security attacks. If an attack is found, in that case, Scyther provides a trace that shows an exact sequence of actions against vulnerabilities. The analysis verified secrecy and authentication properties through the Niagree, Nisynch, Alive, Weakagree, and Secret claims. The Scyther simulation results show that there is no attack

Scyther results : verify					
Claim				Status	Comments
Proposed MQTTClient	Proposed MQTTClient1	Secret HMAC(Concat(clicstate,counter))	OK	No attacks within bound	
	Proposed MQTTClient2	Niagree	OK	No attacks within bound	
	Proposed MQTTClient3	Nisynch	OK	No attacks within bound	
	Proposed MQTTClient4	Alive	OK	No attacks within bound	
	Proposed MQTTClient5	Weakagree	OK	No attacks within bound	
MQTTBroker	Proposed MQTTBroker1	Secret HMAC(Concat(clicstate,counter))	OK	No attacks within bound	
	Proposed MQTTBroker2	Niagree	OK	No attacks within bound	
	Proposed MQTTBroker3	Nisynch	OK	No attacks within bound	
	Proposed MQTTBroker4	Alive	OK	No attacks within bound	
	Proposed MQTTBroker5	Weakagree	OK	No attacks within bound	
Done.					

Fig. 6. Scyther simulation results of Phase 2.

within the bounds as shown in Figs. 5 and 6. Hence, we conclude that our proposed protocol is secure against known security attacks.

IX. CONCLUSION AND FUTURE WORK

With the rapid growth of the Internet of Things, the MQTT protocol has become widely used for lightweight and scalable messaging in IoT applications. This article provides a detailed review of the various security challenges and vulnerabilities that MQTT faces. To address these issues, we propose an integrated approach in the form of a framework called SAF. SAF integrates two key components: the SAF authentication protocol and an LLM-based IDS. Our analysis shows that SAF is a holistic approach that offers a secure and reliable way to automate threat detection in the MQTT environment and authenticate MQTT communications, as it segregates the main roles of authentication, restrictions implementation, and monitoring. To ensure that it can be implemented efficiently on resource-constrained IoT devices, SAF is designed to avoid the use of heavy computational processes like public key cryptography in its authentication protocol. Furthermore, the SAF authentication protocol demonstrates efficiency primarily in terms of computational operations. The Scyther simulation results of our proposed protocol demonstrate that there is no security attacks are possible within bounds. This makes SAF suitable for IoT environments where computational resources and energy efficiency are crucial considerations. The LLM-based IDS is a new concept introduced for MQTT systems with distinctive characteristics that enable it to detect advanced context-based attacks more efficiently. Future work will include implementing the SAF approach in a large-scale MQTT environment and providing verification and recommendations for its performance advantages.

In the current study, the experimental section focuses on the conceptual feasibility of the proposed SAF through protocol design and formal security validation using Scyther simulation. The scope and aim of the work are limited to establishing correctness and demonstrating the effectiveness of the integration of state-based authentication with LLM-based IDS, instead of providing an empirical LLM-based IDS performance evaluation. However, we acknowledge that this limits the depth of experimental analysis. In future work, we plan to implement the SAF framework in an MQTT testbed to evaluate the performance metrics, which will provide a comprehensive quantitative analysis of SAF's efficiency, security, and practical applicability.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers and the associate editor for their valuable feedback on the article.

REFERENCES

- [1] F. Chen, Y. Huo, J. Zhu, and D. Fan, "A review on the study on MQTT security challenge," in *Proc. IEEE Int. Conf. Smart Cloud (SmartCloud)*, Nov. 2020, pp. 128–133.
- [2] M. Ion, "Security of publish/subscribe systems," Università di Trento, Trento, Italy, Tech. Rep. 08.1, 2013.
- [3] N. Jeyanthi, "Internet of Things (IoT) as interconnection of threats (IoT)," in *Security and Privacy in Internet of Things (IoTs)*. Boca Raton, FL, USA: CRC Press, 2016, pp. 23–42.
- [4] H. Wong and T. Luo, "Man-in-the-middle attacks on MQTT-based IoT using BERT based adversarial message generation," in *Proc. KDD AIoT Workshop*, vol. 8, 2020, pp. 130–145.
- [5] P. S. Akshatha, S. M. D. Kumar, and K. R. Venugopal, "MQTT implementations, open issues, and challenges: A detailed comparison and survey," *Int. J. Sensors, Wireless Commun. Control*, vol. 12, no. 8, pp. 553–576, Oct. 2022.
- [6] M. H. Widiyanto and R. Ranny, "Message querying telemetry transfer on IoT applications to enhance technology: A systematic review," *Int. J. Reconfigurable Embedded Syst. (IJRES)*, vol. 11, no. 3, p. 265, Nov. 2022.
- [7] A. J. Hintaw, S. Manickam, S. Karuppayah, and M. F. Aboalmaaly, "A brief review on MQTT's security issues within the Internet of Things (IoT)," *J. Commun.*, vol. 14, no. 6, pp. 463–469, 2019.
- [8] S. R. Akbar, K. Amron, H. Mulya, and S. Hanifah, "Message queue telemetry transport protocols implementation for wireless sensor networks communication—A performance review," in *Proc. Int. Conf. Sustain. Inf. Eng. Technol. (SIET)*, Nov. 2017, pp. 107–112.
- [9] M. B. Yassein, M. Q. Shatnawi, S. Aljwarneh, and R. Al-Hatmi, "Internet of Things: Survey and open issues of MQTT protocol," in *Proc. Int. Conf. MIS (ICEMIS)*, May 2017, pp. 1–6.
- [10] O. Open. (2020). *Oasis*. [Online]. Available: <https://www.oasis-open.org/org>
- [11] B. Mishra and A. Kertesz, "The use of MQTT in M2M and IoT systems: A survey," *IEEE Access*, vol. 8, pp. 201071–201086, 2020.
- [12] *Mqtt Version 5.0*, O. Standard, Woburn, MA, USA, 2019.
- [13] N.-W. Lo and S.-H. Hsu, "A secure IoT firmware update framework based on MQTT protocol," in *Proc. Int. Conf. Inf. Syst. Archit. Technol. (ISAT)*. Cham, Switzerland: Springer, Sep. 2020, pp. 187–198.
- [14] Y. Im and M. Lim, "E-MQTT: End-to-end synchronous and asynchronous communication mechanisms in MQTT protocol," *Appl. Sci.*, vol. 13, no. 22, p. 12419, Nov. 2023.
- [15] X. Wu and N. Li, "Improvements of MQTT retain message storage mechanism," in *Proc. 2nd IEEE Adv. Inf. Manage., Communicates, Electron. Autom. Control Conf. (IMCEC)*, May 2018, pp. 957–961.
- [16] R. Chen, X. Du, J. Hu, and T. Song, "Blockchain-based MQTT communication access control scheme for the Internet of Things," in *Proc. 2nd Int. Conf. Electron. Inf. Technol. (EIT)*, vol. 8, 2023, p. 159.
- [17] N. S. Alotaibi, H. I. Sayed Ahmed, S. O. M. Kamel, and G. F. Elkabbany, "Secure enhancement for MQTT protocol using distributed machine learning framework," *Sensors*, vol. 24, no. 5, p. 1638, Mar. 2024.
- [18] G. A. Paul, Y. Jagnani, and P. Supraja, "Improving fault tolerance and tackling broker failure in MQTT through blockchain," in *Proc. 3rd Int. Conf. Adv. Electr., Comput., Commun. Sustain. Technol. (ICAECT)*, Jan. 2023, pp. 1–6.
- [19] L. Chen, S. Vidalis, and S. Yang, "A trust-based approach for data sharing in the MQTT environment," in *Proc. 20th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2023, pp. 1–5.
- [20] A. P. Haripriya and K. Kulothungan, "Secure-MQTT: An efficient fuzzy logic-based approach to detect DoS attack in MQTT protocol for Internet of Things," *EURASIP J. Wireless Commun. Netw.*, vol. 2019, no. 1, p. 90, Dec. 2019.
- [21] D. Dikii, S. Arustamov, and A. Grishentsev, "DoS attacks detection in MQTT networks," *Indonesian J. Electr. Eng. Comput. Sci.*, vol. 21, no. 1, pp. 601–608, Jan. 2021.
- [22] N. F. Syed, Z. Baig, A. Ibrahim, and C. Valli, "Denial of service attack detection through machine learning for the IoT," *J. Inf. Telecommun.*, vol. 4, no. 4, pp. 482–503, Oct. 2020.
- [23] M. Al-Fayoumi and Q. Abu Al-Haija, "Capturing low-rate DDoS attack based on MQTT protocol in software defined-IoT environment," *Array*, vol. 19, Sep. 2023, Art. no. 100316.

- [24] L. M. da Silva et al., "Development of an efficiency platform based on MQTT for UAV controlling and DoS attack detection," *Sensors*, vol. 22, no. 17, p. 6567, Aug. 2022.
- [25] R. D. Paz, A. Sehovic, D. M. Cook, and L. Armstrong, "A novel approach to resource starvation attacks on message queuing telemetry transport brokers," in *Proc. 4th Int. Conf. Inf. Technol., Inf. Syst. Electr. Eng. (ICITISEE)*, Nov. 2019, pp. 150–154.
- [26] J. Chen, Z. Chang, W. Guo, and X. Guo, "Resource allocation and computation offloading for wireless powered mobile edge computing," *Sensors*, vol. 22, no. 16, p. 6002, Aug. 2022.
- [27] A. Viswanathan, "Analysis of power consumption of the mqtt protocol," Ph.D. dissertation, School Inf. Sci. Telecommun., Univ. Pittsburgh, Pittsburgh, PA, USA, 2017.
- [28] F. Palmese, A. E. C. Redondi, and M. Cesana, "Adaptive quality of service control for MQTT-SN," *Sensors*, vol. 22, no. 22, p. 8852, Nov. 2022.
- [29] M. Matic, M. Antić, S. Ivanovic, and I. Pap, "Scheduling messages within MQTT shared subscription group in the clustered cloud architecture," in *Proc. 28th Telecommun. Forum (TELFOR)*, 2020, pp. 1–4.
- [30] W. Pipatsakulroj, V. Visoottiviset, and R. Takano, "MuMQ: A lightweight and scalable MQTT broker," in *Proc. IEEE Int. Symp. Local Metrop. Area Netw. (LANMAN)*, Jun. 2017, pp. 1–6.
- [31] P. Colombo, E. Ferrari, and E. D. Tümer, "Regulating data sharing across MQTT environments," *J. Netw. Comput. Appl.*, vol. 174, Jan. 2021, Art. no. 102907.
- [32] M. Markovic and P. Edwards, "Enhancing transparency of MQTT brokers for IoT applications through provenance streams," in *Proc. 6th Int. Workshop Middleware Appl. Internet Things*, 2019, pp. 17–20.
- [33] M. Fischer, D. Kümper, and R. Tönjes, "Towards improving the privacy in the MQTT protocol," in *Proc. Global IoT Summit (GloTS)*, 2019, pp. 1–6.
- [34] M. Friesen, G. Karthikeyan, S. Heiss, L. Wisniewski, and H. Trsek, "A comparative evaluation of security mechanisms in DDS, TLS and DTLS," in *Kommunikation Und Bildverarbeitung in Der Automation*. Cham, Switzerland: Springer, 2020, pp. 201–216.
- [35] A. Luykx and K. G. Paterson, "Limits on authenticated encryption use in TLS," *Cryptol. ePrint Arch.*, Paper 2024/051, Jan. 2024, pp. 1–8.
- [36] M. Scott, "On tls for the Internet of Things, in a post quantum world," *Cryptol. ePrint Arch.*, Paper 2023/095, Oct. 2023, pp. 1–19.
- [37] R. J. Robinson, "Insights on cloud security management," *Cloud Comput. Data Sci.*, vol. 4, no. 2, pp. 212–222, Jul. 2023.
- [38] J. Bozhko, Y. Hanna, R. Harrilal-Parchment, S. Tonyali, and K. Akkaya, "Performance evaluation of quantum-resistant TLS for consumer IoT devices," in *Proc. IEEE 20th Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2023, pp. 230–235.
- [39] F. Bozkurt, M. Kara, M. A. Aydin, and H. H. Balik, "Exploring the vulnerabilities and countermeasures of SSL/TLS protocols in secure data transmission over computer networks," in *Proc. IEEE 12th Int. Conf. Intell. Data Acquisition Adv. Comput. Systems: Technol. Appl. (IDAACS)*, Sep. 2023, pp. 400–407.
- [40] O. Ivanov, V. Ruzhentsev, and R. Oliynykov, "Comparison of modern network attacks on TLS protocol," in *Proc. Int. Sci.-Practical Conf. Problems Infocommun. Sci. Technol. (PIC S&T)*, Oct. 2018, pp. 565–570.
- [41] Ö. Yerlikaya and G. Dalkılıç, "Authentication and authorization mechanism on message queue telemetry transport protocol," in *Proc. 3rd Int. Conf. Comput. Sci. Eng. (UBMK)*, 2018, pp. 145–150.
- [42] M. S. Harsha, B. M. Bhavani, and K. R. Kundhavai, "Analysis of vulnerabilities in MQTT security using shodan API and implementation of its countermeasures via authentication and ACLs," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Sep. 2018, pp. 2244–2250.
- [43] R. R. Pahlevi, P. Sukarno, and B. Erfianto, "Implementation of event-based dynamic authentication on MQTT protocol," *Jurnal Rekayasa Elektrika*, vol. 15, no. 2, pp. 125–133, Sep. 2019.
- [44] F. Buccafurri, V. De Angelis, and R. Nardone, "Securing MQTT by blockchain-based OTP authentication," *Sensors*, vol. 20, no. 7, p. 2002, Apr. 2020.
- [45] S. Pal and Z. Jadidi, "Protocol-based and hybrid access control for the IoT: Approaches and research opportunities," *Sensors*, vol. 21, no. 20, p. 6832, Oct. 2021.
- [46] H. Kurdi and V. Thayanathan, "Authentication mechanisms for IoT system based on distributed MQTT brokers: Review and challenges," *Proc. Comput. Sci.*, vol. 194, pp. 132–139, Jan. 2021.
- [47] P. Han, Z. Zhang, S. Ji, X. Wang, L. Liu, and Y. Ren, "Access control mechanism for the Internet of Things based on blockchain and inner product encryption," *J. Inf. Secur. Appl.*, vol. 74, May 2023, Art. no. 103446.
- [48] A. Bashir and A. Hussain Mir, "Securing communication in MQTT enabled Internet of Things with lightweight security protocol," *EAI Endorsed Trans. Internet Things*, vol. 3, no. 12, p. e1, Oct. 2017.
- [49] A. Varma and S. UniKrishnan, "Effect of payload security in MQTT protocol over transport and application layer," *Proc. IOP Conf., Mater. Sci. Eng.*, vol. 1166, no. 1, 2021, Art. no. 012019.
- [50] T.-L. Liao, H.-R. Lin, P.-Y. Wan, and J.-J. Yan, "Improved attribute-based encryption using chaos synchronization and its application to MQTT security," *Appl. Sci.*, vol. 9, no. 20, p. 4454, Oct. 2019.
- [51] P. Fanfara, E. Danková, and M. Dufala, "Usage of asymmetric encryption algorithms to enhance the security of sensitive data in secure communication," in *Proc. IEEE 10th Int. Symp. Appl. Mach. Intell. Informat. (SAMI)*, Jan. 2012, pp. 213–217.
- [52] A. Fauzan, P. Sukarno, and A. A. Wardana, "Overhead analysis of the use of digital signature in MQTT protocol for constrained device in the Internet of Things system," in *Proc. 3rd Int. Conf. Comput. Informat. Eng. (IC2IE)*, Sep. 2020, pp. 415–420.
- [53] K. Jamshaid and P. A. S. Ward, "Experiences using gateway-enforced rate-limiting techniques in wireless mesh networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, Mar. 2007, pp. 3725–3730.
- [54] S. Kumar and D. K. Vasthimal, "Cardinality based rate limiting system for time-series data," in *Proc. 13th Int. Conf. Cloud Comput.*, Honolulu, HI, USA, Sep. 2020, pp. 250–260.
- [55] I. Nawab and S. G. Kulkarni, "Toward next-generation distributed rate-limiters," in *Proc. IEEE/ACM 23rd Int. Symp. Cluster, Cloud Internet Comput. Workshops (CCGridW)*, May 2023, pp. 354–356.
- [56] A. La Marra, F. Martinelli, P. Mori, A. Rizzo, and A. Saracino, "Introducing usage control in MQTT," in *Proc. ESORICS Int. Workshops Comput. Security*, Sep. 2017, pp. 35–43.
- [57] E. Ciklabakkal, A. Donmez, M. Erdemir, E. Suren, M. K. Yilmaz, and P. Angin, "ARTEMIS: An intrusion detection system for MQTT attacks in Internet of Things," in *Proc. 38th Symp. Reliable Distrib. Syst. (SRDS)*, Oct. 2019, pp. 369–3692.
- [58] F. Mosaiyebzadeh, L. G. Araujo Rodriguez, D. M. Batista, and R. Hirata, "A network intrusion detection system using deep learning against MQTT attacks in IoT," in *Proc. IEEE Latin-Amer. Conf. Commun. (LATINCOM)*, Nov. 2021, pp. 1–6.
- [59] C. Birkinshaw, E. Rouka, and V. G. Vassilakis, "Implementing an intrusion detection and prevention system using software-defined networking: Defending against port-scanning and denial-of-service attacks," *J. Netw. Comput. Appl.*, vol. 136, pp. 71–85, Jun. 2019.
- [60] M. Husnain et al., "Preventing MQTT vulnerabilities using IoT-enabled intrusion detection system," *Sensors*, vol. 22, no. 2, p. 567, Jan. 2022.
- [61] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis, and X. Bellekens, "Machine learning based IoT intrusion detection system: An MQTT case study (MQTT-IoT-IDS2020 dataset)," in *Proc. Int. Netw. Conf.*, 2020, pp. 73–84.
- [62] M. A. I. Mahmood, F. Ashab, M. Saifuzzaman Sohan, M. Hedayetul Islam Chy, and M. F. Kader, "LLM-enhanced security framework for IoT network: Anomaly detection and malicious devices identification," *IEEE Access*, vol. 13, pp. 168405–168419, 2025.
- [63] S. Elouardi, A. Motii, M. Jouhari, A. Nasser Hassane Amadou, and M. Hedabou, "A survey on hybrid-CNN and LLMs for intrusion detection systems: Recent IoT datasets," *IEEE Access*, vol. 12, pp. 180009–180033, 2024.
- [64] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schlaffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *J. Cryptol.*, vol. 34, no. 3, pp. 1–42, Jul. 2021.
- [65] A. M. Kaiser and G. W. Hoiness, "Throughput of ascon compared with popular IoT encryption algorithms," *Mil. Cyber Affairs*, vol. 8, no. 1, p. 3, 2025.
- [66] M. Shariq et al., "Anonymous and reliable ultralightweight RFID-enabled authentication scheme for IoT systems in cloud computing," *Comput. Netw.*, vol. 252, Oct. 2024, Art. no. 110678.
- [67] M. Shariq, N. Jamil, G. S. Rawat, S. A. Chaudhry, M. Masud, and A. Cangelosi, "An anonymous and privacy-preserving lightweight authentication protocol for secure communication in UAV-assisted IAV networks," *Comput. Commun.*, vol. 239, Jul. 2025, Art. no. 108192.
- [68] M. Shariq et al., "Design of provably secure and lightweight authentication protocol for unmanned aerial vehicle systems," *Comput. Commun.*, vol. 228, Dec. 2024, Art. no. 107971.